

Web3 支付：业务通道、托管增信、对账兜底

gomall 把”第三方支付吃不下订单”接回来：跨境 / 加密原生 / 大额托管

gomall · 业务侧 deck

业务定位：Web3 不是新潮，是”补一块法币吃不了的市场”

业务码 + 客服话术：Web3 通道对外暴露的 4 类问题

对账事故 SOP：链上事件丢 / 重 / 撤销三大场景

钱包签名：用户私钥从不离手

Escrow 合约：业务承诺的最低契约

业务边界与合规：gomall 在 Web3 通道里”不做什么”

各角色视角 + 路线图

业务定位：Web3 不是新潮，是”补一块法币吃不下市场”

先讲清楚 gomall 为什么开 Web3 通道

不是为了潮，不是为了讲技术，是为了三类订单：

- **加密原生用户**：钱包里只有 ETH / USDT，没绑卡也没法绑卡（行业数据：DeFi 月活 5-7M 钱包），传统通道直接把他们挡在门外。
- **跨境订单**：买家在东南亚 / 拉美 / 非洲，传统跨境 Visa / SWIFT 收 1-3% 手续费 + T+1 到账，争议解决靠银行流程长达 30-90 天。
- **大额托管交易**：单笔 \$1,000+ 跨境订单，买卖双方互不信任、传统支付牌照机构不愿担保，需要”中介信任”换”代码不变式”。

Web3 通道与法币 / paydown **并存**，不是替换。开新通道 = 拓客，不是减客。

internal/payment/routes.go:14-27 paydown（法币）与 *paydown/crypto*（web3）平行挂载 *README.md:69-80*
业务边界：”不真上链”是显式声明的范围

5 角色视角：Web3 通道对每个角色意味着什么

角色	在 Web3 通道里关心什么	主要痛点
C 端用户	钱安全吗 / 链上钱付了为什么还没到	"pending 半小时是不是丢了"
商家	我什么时候确认到账 / 谁出 gas	T+0 vs 法币 T+1, 但 gas 自付有疑虑
运营	开通道能多接多少订单	拓客转化率 / 客单价
客服	用户骂"网络回滚我钱呢"咋办	需要 reorg / pending 的话术
SRE	链是不可靠通道, 掉块怎么办	listener catch-up / 退避 / 多源

业务承诺要分角色给: 用户要的是"钱在哪我能查", 商家要的是"什么时候 release", 客服要的是"按业务码读话术不掉链子", SRE 要的是"链断 30 分钟主流程不挂"。

README.md:21-29 5 角色业务全景表

业务价值定量：开一条 Web3 通道相当于开三个用户群

新用户群	行业体量参考	gomall 接入收益
DeFi 原生	月活 5-7M 钱包	钱包内资产可直接付款，零卡线
跨境（拉美 / 东南亚 / 非洲）	跨境电商 GMV 年增 13%	1-3% 手续费 → gas 固定，T+
大额 B2B 托管	单笔 \$1k-\$50k	中介信任 → 合约不变式，争议

数字不用记牢，记住一句：**Web3 不是替代第三方支付，是补齐法币吃不下订单形态。**法

币通道仍跑 /paydown (deck 05)，Web3 通道跑 /paydown/crypto，两条线并行。

业务边界对外坦白：MVP 阶段默认不接 RPC (WEB3_RPC_URL 不设 listener 不启动)，合约 + binding 已就绪。*internal/payment/routes.go:14-27* 法币 + Web3 两条支付通道并行 *README.md:74-80*
不真上链是显式业务边界

跨境用户旅程：东南亚买家在 gomall 下单的真实链路

角色： 印尼买家 Andi，钱包里有 USDT，要买一台 \$280 的中国卖家相机。

- **法币通道（不可行）：** 当地银行卡跨境刷卡费 2.5% + 汇率损耗 1%；到账 T+1；纠纷 30-90 天。
- **Web3 通道：** 钱包签名 → fundWithOrderID 把 USDT 锁进 escrow → 1-2 分钟链上确认 → 商家发货 → Andi 收到货确认 release → 卖家拿到 USDT。
- **时间窗对比：** 法币通道 1 周资金锁定 + 1% 手续费；Web3 通道 1-2 分钟到账 + gas \$0.05-\$0.30 (L2)。
- **争议处理：** 传统 chargeback 走 Visa 流程 30-60 天，结果靠平台 / 银行裁；Web3 escrow 走 dispute 状态 → arbiter 在链上裁决，证据是不可篡改的链上事件。

业务收益： 平台多接一类跨境订单（DeFi 钱包用户 + 跨境无卡用户），手续费收入从买家结算端转到链上 gas（用户自付），平台净抽成空间反而提升。contracts/Escrow.sol:68-82 fund;

:85-107 release/refund/dispute 状态出口 internal/payment/routes.go:14-27 法币 + Web3 通道并行

运营视角：开 Web3 通道的转化数据怎么追

运营要回答两个业务问题：

- **Q1：开 Web3 通道带来多少新用户？**

- 埋点：注册流里”绑定钱包”按钮的点击率 / 完成率；
- 看板：每日新增钱包绑定用户 / Web3 支付 GMV / Web3 通道 ARPU；
- 决策：如果 Web3 通道 GMV < 平台总 GMV 的 1%，是否关闭通道节约 RPC 成本？

- **Q2：Web3 通道的转化漏斗在哪卡？**

- 漏斗：进入支付页 → 选 Web3 → 钱包签名成功 → 链上 fund → 6 块确认；
- gomall 当前实现：每步都写 outbox 事件 (pending / signed / confirmed)，运营 BI 可拉链跑数；
- 典型卡点：钱包签名 → 链上 fund 之间的”用户钱包余额不够 gas”，需要 UI 提示先充 gas。

internal/payment/service_crypto.go:129-148 outbox 事件埋点是 BI 数据源 service/web3/listener.go:303-311

listener 写入 web3.payment.confirmed 是漏斗末段

Escrow 业务价值：信任锚从” 银行” 换成” 可审计代码”

Escrow 不是技术炫技，是回答一个业务问题：买卖双方互不信任的大额订单，怎么办？

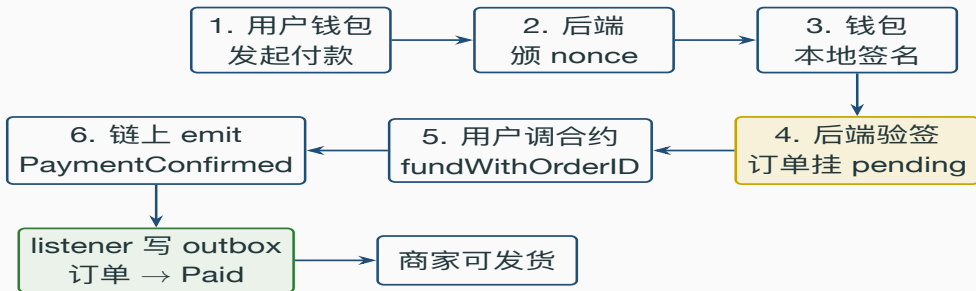
- **传统答案：**找第三方（支付牌照机构 / 平台）做担保。代价 = 1-3% 手续费 + 平台跑路风险 + 监管套利空间。
- **Web3 答案：**把” 担保人” 换成一段第三方审计过的代码，资金锁在合约里，合约只接受三种动作——买家确认收货 → 给商家；买家退货 → 退给买家；争议 → 仲裁人裁决。
- **业务承诺等价：**合约的” 不变式”（buyer / seller / amount 部署后不可改、Created→Funded 只能一次、终态不可逆）= 平台对外承诺的最低契约。

一句话总结：平台不再说” 相信我”，而是说” 相信你能读懂的代码”。

contracts/Escrow.sol:13-19 状态机；:68-82 fund/fundWithOrderID；:85-107 release/refund/dispute

contracts/README.md:1-36 合约职责与不变式条款

端到端 6 步业务流：钱 → 合约 → 订单



业务保障：钱没进合约前，订单永远 pending（用户不会“先付款后丢单”）；钱进合约后，listener 兜底推进（即便后端那一刻挂了，链上证据永远在）。这是 Web3 比传统支付强的核心一点——**业务状态有“链”做单点证据源**，不依赖平台自己的 DB。

internal/payment/service_crypto.go:48-84 IssueNonce 颁发 internal/payment/service_crypto.go:86-160

VerifyAndPark 验签 + outbox service/web3/listener.go:177-230 链上事件兜底推进订单

商家收款体验：什么时候算”到账”

商家最关心的不是技术细节，是”我什么时候可以发货”：

确认数	时间窗口	商家可以做什么
0 块 (mempool)	< 30s	不可发货，钱还没上链
1 块	30–60s	可显示”已上链”，小额订单 (< \$100) 可发货 业务
6 块	≈ 90s	中额订单 (\$100–\$1k) 可发货，抗常规 reorg
32 块 (1 epoch)	6.4 分钟	大额订单 (> \$1k) finalize 后才可 release

规则：金额越大 N 越大，是商家自己的风控阈值 (gomall 默认值在配置文件里，商家后台可调，路线图阶段)。商家提现到自己钱包的 gas 由商家自付 (用户付款 gas 由用户自付，平台不补贴，避免被刷)。

用户取消订单流程：钱已进 escrow 但商家未发货，买家点取消 → arbiter 触发 refund → 链上退回买家钱包 → listener 写 `web3.payment.refunded` → 订单状态 Refunded。

`contracts/Escrow.sol:85–107` release / refund / dispute 三个出金口 `service/web3/listener.go:296–301` 事件级幂等保证 refund 不会重复推进

用户体验：链是异步的，UX 必须 ticket 化



上”秒级到账”是反物理的。业务侧 UX 走 **ticket 模式**：钱包签名完成 → 后端立即返回 W3-PENDING + ticket → 前端轮询订单状态 → 1 确认才显示”已上链”，6 确认才允许商家发货。

客服话术对齐：W3-PENDING 不是失败，是”等待区块确认（约 30-60s）”，要让用户知道这是正常状态，不是异常状态。这一点 gomall 与 deck 05 法币的”立即到账”承诺刻意不一致。*internal/payment/service_crypto.go:156-159* 立即返回 pending 状态 *service/web3/listener.go:177-230* head 推进，UI 状态由 outbox 驱动

业务码 + 客服话术：Web3 通道对外 暴露的 4 类问题

Web3 业务码 + 客服话术（核心 4 条）

业务码	含义	客服话术
W3-NONCE-EXPIRED	nonce 5min 过期或已用	” 请重新发起支付”
W3-SIG-INVALID	签名地址不匹配	” 请用绑定钱包重试”
W3-PENDING	链上未确认	” 等待区块确认（约 30–60s） ”
W3-REORG	链上 reorg 撤销	” 网络回滚，已自动退款”

客服话术原则：不暴露技术细节，但要**足够区分**。W3-PENDING 与 W3-NONCE-EXPIRED 表面像

（都是” 等”），处理完全不同：前者继续等链确认，后者必须重发 nonce 重签。

与 gomall 已有业务码体例一致（参 `pkg/e/code.go`）：30001/30002 鉴权类、50001 OSS、60002 幂等、70001 限流、70002 熔断。Web3 这套 W3-* 是字符串码（不是数字），因为它们要给前端钱包弹窗和客服话术读，不是给 SDK 解析。`pkg/e/code.go:34–58` 业务码体例：数字码

用于 SDK，字符串码用于钱包弹窗 `repository/cache/web3.go:29–33` `ErrWeb3NonceMissing /`

`ErrWeb3NonceMismatch`

客服话术三场景：用户最常问的三类问题

场景 1：用户问“链上交易 pending 半小时了”

- 客服话术：先确认链 ID (mainnet 还是 L2)，mainnet 拥堵高峰 pending 1-2 小时正常；L2 超过 5 分钟才算异常。
- SRE 后台动作：查 `web3:nonce` 是否已消费（已消费 = 签名通过）+ 查 `listener` 日志该 `txhash` 是否收到；若 `nonce` 已消费但 `txhash` 收不到 → 用户钱包侧 tx 还没广播或被 drop。

场景 2：用户问“我钱付了为什么订单还没确认”

- 客服话术：付款分两段——钱包签名（瞬时）+ 链上转账（30-60s）。让用户在钱包里查 tx 状态。
- SRE 后台：grep `outbox` 中 `web3.payment.pending` 找到该订单；若有 pending 但没 `confirmed` → 用户钱包侧的 tx 失败了或还没确认。

场景 3：用户问“我能退回原通道吗（法币卡）”

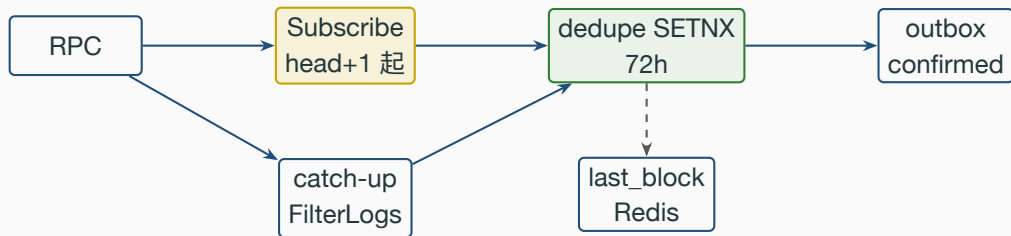
对账事故 SOP：链上事件丢 / 重 / 撤销三大场景

链是不可靠通道：3 类事故 + SRE 处置 SOP

事故	业务后果	SRE 处置
listener 重启	漏掉停机期间事件	进程启动自动 catchUp(last+1, head) 回放, 无需人工
RPC 节点断	短期收不到事件	退避重连 1s→60s 封顶, 超时仍未恢复值班介入
链上 reorg	已上链的 tx 被撤销	listener 收到 log.Removed 跳过; 订单未推进过 = 无事; 已推进 → arbiter 介入 refund
重复事件	同一笔订单推进两次	web3:event:{txhash}:{logIdx} SETNX 72h 兜底, 重复 直接 short_circuit

业务承诺 (SLO):

listener 三项基本功：业务承诺背后的实现



- **重连**：RPC 短暂断 = 业务延迟感知，不掉单；指数退避 1s→60s 封顶，连上 reset。
- **catch-up**：进程重启或 RPC 切换时拿 last_block 批量 FilterLogs 回放——这是 gomall 对外“不掉单”承诺的底牌。
- **幂等**：每条事件 `web3:event:{txhash}:{logIdx} SETNX 72h`；reorg / 重连 / 重启全靠它兜底，业务侧无感。

容错矩阵：异常 vs 业务影响 vs 兜底

异常	listener 行为	业务影响
RPC 断连	backoff 1s → 60s 指数重连	短暂延迟，不掉单
进程重启	loadLastBlock 后 catchUp(last+1, head)	< 5min 内追完，不掉单
log.Removed	reorg 撤销，跳过	SETNX 兜底，订单不二次推进
重复事件	SETNX 72h key 已存在 return	不重复推进订单
RPC 慢	Subscribe buffer=64 不阻塞主流程	log 缓冲，不丢
RPC 全断	listener 静默，订单挂 pending	法币通道仍可用

业务承诺的复用：listener 写入的 `web3.payment.confirmed` 与法币的 `order.paid` 走同一条 outbox 总线（deck 04 / deck 11 同款），财务对账可以拿到**统一格式**的事件，按 `order_id` 维度 join，不需要为 Web3 单独建对账链路。 `service/web3/listener.go:155-175 run()` 退避重连；`:218-221 reorg` 跳过 `service/web3/listener.go:296-301 dedupeKey 72h`；`:303-311 outbox.Insert`

真实事故还原：reorg 把已发货订单的钱”撤回”了

场景：商家在 1 块确认时就发货（小额订单的默认风控阈值），10 分钟后链 reorg 把那条 tx 撤销了。

- **listener 视角：**收到 `log.Removed=true` 的同一条 `PaymentConfirmed`，跳过不再写 `outbox`。
- **订单视角：**状态已经因为之前那次 `confirm` 推进到 `Paid`, `goods` 已发；reorg 后 `listener` 不会再推一次（`SETNX 72h` 兜底），但已发生的状态推进不会自动回滚。
- **SRE 处置 SOP：**
 - Step 1: 值班按 `alert` 找到该订单 ID + `txhash`；
 - Step 2: 链上确认 tx 真的没回归（等 6 块再确认一次）；
 - Step 3: `arbiter` 调 `refund` 把 `escrow` 里的钱（如果还在）退给买家；
 - Step 4: 商家走链下追款；这单算作平台损失（这正是为什么大额订单要等 6+ 确认才发货）。

对外承诺：风控阈值（确认数）由商家自己设；gomall 默认小额 1 块 / 中额 6 块 / 大额 32 块，商家可调。商家承担”提前发货”的风险，平台不兜底。这是 Web3 通道与法币通道的

真实事故还原：listener 漏事件怎么人工补

场景：RPC 节点配置错误连续 30 分钟没收到事件，期间链上有 12 笔 PaymentConfirmed。

- 发现：监控 alert 触发，“listener 60s 内未消费任何 head”。
- 标准处置：listener 重启 → loadLastBlock 拿到 30 分钟前的块号 → catchUp(last+1, head) 用 FilterLogs 批量回放 → 全部 12 笔补齐。
- 异常处置（极端情况）：last_block 这个 Redis key 也丢了 → 人工指定回放起点。SRE 后台脚本：从订单库找最近 30min 的 Web3 pending 订单的最早 created_at → 找对应区块号 → 手动调 listener admin 接口（路线图阶段）重置 last_block → 重启服务。

这种“补”是业务承诺的兜底层：哪怕缓存全丢、进程多次崩溃，只要链上事件还在，所有 PaymentConfirmed 都能复原。这是 **Web3 强于传统支付的地方**——链是单点真源（single source of truth），传统支付一旦 webhook 丢了就只能找对方重发。

service/web3/listener.go:232-248 catchUp 批量回放 service/web3/listener.go:328-352 loadLastBlock /

钱包签名：用户私钥从不离手

业务约束：为什么后端永远不能拿到私钥

业务侧最低承诺：用户私钥 = 用户钱包的全部，gomall 任何环节都不接触。

- **法律意义：**私钥被泄漏 = 平台无法对外解释（监管 / 用户 / 媒体），合规风险无上限。
- **业务意义：**用户对 Web3 平台的信任完全建立在“不托管私钥”上——一旦托管，回到传统支付，Web3 的全部价值消失。
- **技术答案：**后端只发“消息模板”，签名发生在钱包客户端内（MetaMask / WalletConnect / 硬件钱包），后端用 ecrecover 反推地址校验。

对应客服话术：“您的私钥永远在您自己的钱包里，gomall 不存、不传、不知道”。这一句必须能直接念给用户，是 Web3 通道的对外底线。

业务边界声明：gomall 不做钱包托管（没有平台钱包），不做币种兑换（不接 DEX），不做跨链桥（避免 \$25B+ 桥被盗的行业风险），不做 NFT 票据，不发自己的 stablecoin。

pkg/web3/signature/verify.go:1-13 包注释设计意图 README.md:69-80 业务边界明确列出

签名消息模板：把“我在给 gomall 付款”钉在用户眼前

gomall:paydown:order={id}:nonce={n}:chain={cid}



EIP-191: keccak256("\x19Ethereum Signed Message:\n<len>" + msg)



ecrecover(hash, sig) → address vs 钱包地址

业务考量：

- 前缀 **gomall:paydown:**：钱包弹窗显示给用户看，是反钓鱼最后一道防线——用户看到不是 gomall 的前缀，应当拒签。
- **chainID** 入消息：防一条签名在 mainnet / L2 互通造成跨链重放（不同链的资产用同一签名重复扣两次）。
- **orderId + nonce**：绑定订单 + 防同链重放，每订单一次签名，签了就消失。
- **v 值归一**：钱包返回 27/28，go-ethereum 还原 pubkey 要 0/1，必须归一。

VerifyPersonalSign: EIP-191 + ecrecover 12 行

Listing 1: *pkg/web3/signature/verify.go:31-67*

```
31 func VerifyPersonalSign(addr string, msg []byte, sig []byte) (bool, error) {
32     if len(sig) != personalSigLen { return false, fmt.Errorf("...") }
33     if !common.IsHexAddress(addr) { return false, errors.New("...") }
34     rsv := make([]byte, personalSigLen)
35     copy(rsv, sig)
36     switch rsv[64] {
37     case 27, 28: rsv[64] -= 27
38     case 0, 1: // already normalized
39     default: return false, fmt.Errorf("非法 v 值: %d", rsv[64])
40     }
41     hash := personalSignHash(msg) // EIP-191 prefix + keccak256
42     pubKey, err := crypto.SigToPub(hash, rsv) // ecrecover
43     if err != nil { return false, err }
44     recovered := crypto.PubkeyToAddress(*pubKey)
45     expected := common.HexToAddress(addr)
46     return bytes.Equal(recovered.Bytes(), expected.Bytes()), nil
47 }
```

IssueNonce + VerifyAndPark: 业务主流程拼图

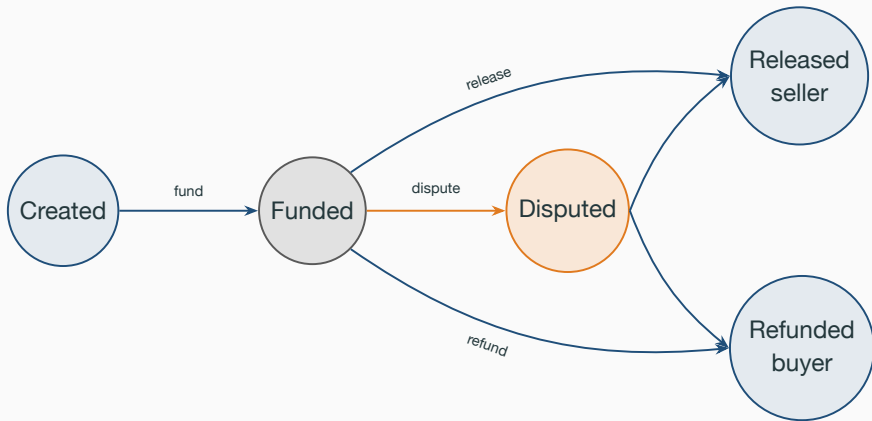
Redis key `web3:nonce:{userID}:{orderId}` 双维度 TTL 5min; 二次签发自动覆盖; 消费走 Lua GET+DEL 原子; Pending 占位 30min 让前端轮询有锚点。 *repository/cache/web3.go:21-26*
TTL; :60-89 *consumeNonceScript*

Listing 2: *internal/payment/service_crypto.go:49-160* 节选

```
49 // IssueNonce: 校验订单归属 + 状态 + 颁发
50 order, _ := orderpkg.NewOrderDao(ctx).GetOrderById(req.OrderId, u.Id)
51 if order.Type != consts.OrderWaitPay { return errors.New("订单状态非未支付") }
52 nonce, _ := randomNonce()
53 cache.PutWeb3Nonce(ctx, u.Id, req.OrderId, nonce) // SET + 5min TTL
54
55 // VerifyAndPark: 消费 nonce -> 验签 -> 事务内写 outbox
56 if err := cache.ConsumeWeb3Nonce(ctx, u.Id, req.OrderID, req.Nonce); err != nil {
57     return err // -1 missing / -2 mismatch -> W3-NONCE-EXPIRED
58 }
59 msg := []byte(BuildSignMessage(req.OrderID, req.Nonce, req.ChainID))
60 ok, _ := web3sig.VerifyPersonalSign(req.WalletAddr, msg, sigBytes)
61 if !ok { return errors.New("签名与钱包地址不匹配") } // W3-SIG-INVALID
62
```

Escrow 合约：业务承诺的最低契约

Escrow 状态机：5 状态 / 3 角色 / 业务对应



业务不变式（对外承诺）：

- Created → Funded 只能发生一次 → 用户付款不会被重复扣
- 终态不可逆 → 商家收到钱（Released）后平台无法撤回
- buyer / seller / arbiter / amount 部署后不可改 → 合约部署即承诺

合约状态 ↔ 订单状态：业务对账映射

合约状态	订单状态	触发动作
Created	已下单未支付	合约部署完成
Funded (链下)	已支付待发货 商家发货 / 物流	买家 fundWithOrderID 链下流程, 不影响合约
Released	已完成	买家 / 仲裁人 release
Refunded	已退款	卖家 / 仲裁人 refund
Disputed	售后争议	买家或卖家 dispute

业务约定：链上只记”钱在谁那里”，订单详情 / 物流 / 评价 / 售后都在链下。对账由 listener 写入 outbox 总线、与法币通道复用同一套 order.paid / order.refunded 事件型号。 *contracts/README.md:31-36* 合约不变式 *service/web3/listener.go:303-311* outbox.Insert 写入 *web3.payment.confirmed*

金额口径事故：满减订单走链上付款被多扣

业务场景：买家命中“满 100 减 20”活动，下单页显示实付 \$80，订单库里 FinalCents=8000、PromoRuleID 已写入。买家选了 Web3 通道付款。

- **出事的地方：**Web3 路径计费写死 $\text{order.Money} \times \text{order.Num}$ ——这是折前总价 \$100，完全没看 FinalCents。outbox 事件 web3.payment.pending 的 Amount 落了 10000 分。
- **业务后果：**满减优惠静默蒸发，链上据这个金额向买家钱包要 \$100。买家明明看到 \$80，实际被要 \$100，多扣 \$20 ——而且是用户自付 gas 的链上转账，扣错了退起来比法币更麻烦。
- **为什么只 Web3 中招：**法币 /paydown 路径早就按 FinalCents 折后实付扣款，Web3 是后开的通道，漏接了这段口径，成了漏网的那一条。

工程教训：一笔订单两条支付路径（法币 / 链上），金额口径必须强制对齐。任何一条新通道复用订单数据时，都得问一句“折后实付这件事它处理了吗”。

internal/payment/service_crypto.go:132-135 修复前写死折前价的位置 *internal/payment/service.go:79-82* 法币 24/32

路径早已取 FinalCents（对照基准）

修法：两条路径统一取 FinalCents 折后实付

修法：与法币路径一字不差地对齐——命中满减（PromoRuleID 非 0）就取折后实付，否则才回落单价 $\times$ 件数。

Listing 3: *internal/payment/service_crypto.go:128-146 VerifyAndPark* 计费口径

```
128 // 实付口径与余额支付路径保持一致：命中满减时以折后实付为准。
129 totalAmount := order.Money * int64(order.Num) // 折前：单价 * 件数
130 if order.PromoRuleID != 0 { // 命中满减
131     totalAmount = order.FinalCents // 取折后实付
132 }
133 // ... 同一事务内写 outbox ...
134 events.Web3PaymentPending{
135     OrderID: order.ID, Num: order.Num,
136     Amount: totalAmount, // 链上据此扣款的金额
137     WalletAddr: walletAddr, ChainID: req.ChainID,
138 }
```

逐行：

• $\text{order.Money} \times \text{order.Num}$ ：无活动订单的默认口径，单价乘件数。

Listing 4: *contracts/Escrow.sol:74–82 fundWithOrderID*

```
74 function fundWithOrderID(bytes32 _orderId)
75     external payable onlyBuyer inState(State.Created)
76 {
77     if (msg.value != amount) revert WrongAmount(amount, msg.value);
78     orderId = _orderId;
79     state = State.Funded;
80     emit Funded(msg.value);
81     emit PaymentConfirmed(_orderId, msg.sender, msg.value);
82 }
```

业务对应:

- `onlyBuyer + inState(Created)`: 只买家本人在“未支付”态能 fund ——防误付 / 防重付。
- `msg.value != amount`: 链上强校验金额, 少 1 wei 也 revert ——业务侧零信任客户端。
- `orderId = _orderId`: 把链下订单号写进 storage, listener 拿它 join 业务库。

业务边界与合规：gomall 在 Web3 通道里”不做什么”

业务边界（显式声明）：Web3 通道里 gomall 不做的事

诚实列出，对外承诺：

- **不真上链**：合约源码 + Go binding 完整，MVP 阶段默认 WEB3_RPC_URL 不设，listener 不启动；订单仍可签名挂 pending（路线图阶段切真链）。
- **不托管钱包**：用户私钥永远在用户自己的钱包，平台没有“代用户签名”能力。
- **不做币种兑换**：用户拿 ETH 付款，商家收 ETH；想换 USDC 用户自己换。
- **不做跨链桥**：每条链独立部署 escrow，不接 CCIP / LayerZero / Wormhole（行业过去 3 年桥被盗 \$25B+，风险太大）。
- **不发 NFT 票据**：订单凭证仍是链下 DB 行，不上链 NFT。
- **不发自有 stablecoin**：用市场上已有的（USDC / USDT / DAI）。
- **不做 KYC / AML 自家方案**：制裁名单过滤路线图阶段接 Chainalysis / TRM Labs API。

README.md:69–80 业务边界对应章节 service/web3/listener.go:113–117 envRPCURL 未设静默不启动

合规边界：链上身份 vs 实名监管

Web3 合规是 gomall MVP 阶段刻意延后的话题，但路线图必须想清楚：

- **钱包地址 ≠ 实名**：链上地址默认匿名，监管要求（FATF Travel Rule）\$3,000+ 交易必须 KYC。
- **阶梯式 KYC 路线图**：\$100 以下免；\$100-\$3,000 邮箱 + 电话；\$3,000+ 强制身份证 + 人脸。
- **OFAC / 制裁名单**：Tornado Cash 等地址必须拒绝；落地 = 接 Chainalysis / TRM Labs API，验签后写 outbox 前过滤。
- **多签仲裁是合规可审计的关键**：单签 arbiter 无法对监管解释“为什么这笔 release”；2-of-3 多签 + 合规岗签名可留下链上证据。

业务声明：gomall MVP 阶段单笔 \$100 以内、单 EOA arbiter；超过 \$100 或托管总额超 \$10k 强制切 Gnosis Safe 2-of-3 多签，不重新部署 Escrow，只在新订单的 constructor 里换 arbiter 地址。*internal/payment/service_crypto.go:116-124* 验签后 / 写 outbox 前，是合规过滤的天然插入点 *contracts/Escrow.sol:10 arbiter immutable*；*contracts/README.md:102* 多签建议

业务承诺 (SLO): Web3 通道的对外承诺

环节	SLO 目标	当前实测	
nonce 颁发	p99 < 100ms	Redis SET + 5min TTL	
验签 + outbox 写入	p99 < 200ms	DB 事务 + Redis SETNX	压测对
链上事件 → outbox 写入	5min 内	catch-up + dedupe	
listener 重启 catch-up	5min 内	FilterLogs 批量回放	
RPC 全断时主流程可用	100%	法币通道独立, Web3 静默降级	

照 (参 stressTest/REPORT.md): 法币 /paydown 走幂等中间件, 跑出 **50K RPS / p95 2.33ms** (50VU × 15s 同 Idempotency-Key 累计 755,033 次请求 → DB 1 笔订单)。Web3 /paydown/crypto 复用同一套幂等 + 事务 + outbox 体系, 业务路径开销 < 5ms (不含链上等待)。

业务保证的根基: 链是不可靠通道但 outbox 总线可靠。listener 写入 outbox 后, 对账 / 通知 / 状态推进全部走与法币通道相同的 web3.payment.confirmed → order.paid 链路 (deck 11 同款)。stressTest/REPORT.md:34–87 /orders/create 与 /paydown 路径的 RPS / p95 / max 数据

各角色视角 + 路线图

各角色视角：业务侧关心的全图

角色	关心	看哪里
用户	钱安全吗	合约不变式 + 不托管承诺 + 退款流程
商家	何时到账	链上确认数 + release 时机 + gas 自付
客服	链上失败咋办	W3-* 业务码对应处理 SOP
运营	开通道转化率多少	新用户群（DeFi / 跨境 / 大额）数据
SRE	链断了主流程挂吗	静默降级 + listener 重连 + outbox 兜底
法务	合规边界	路线图阶段：KYC + 多签 + 司法管辖
财务	链上链下对账	outbox 统一总线 + channel 标识

务对账拿到**统一格式**事件：channel = fiat / web3-eth / web3-usdc，按 order_id join。**业务边界**：链 / 币 / gas 三选一组合，MVP 推荐 **L2 + ETH 原生币 + 用户自付 gas**，正式运营后逐步上稳定币 + gas 补贴 + 多链路由。service/web3/listener.go:303-311 outbox.Insert 与法币通道复用同一总线 README.md:21-29 5 角色 + 业务全景表

Arbiter 多签升级路线图

阶段	arbiter 形态	适用金额上限
MVP	平台 EOA	等值 \$100 以内
2	2-of-3 多签	等值 \$10,000 以内 (Gnosis Safe)
3	DAO 治理	超大额 / 跨组织

业务驱动:

- MVP 单 EOA 私钥放运营手里, 丢私钥 / 私钥被盗 = 所有 Disputed 订单卡死 + 平台跑路风险。
- 单笔 \$10,000+ 用户不会接受”平台单签拍板”; 监管也不接受。
- 升级关键: **切换不需要重新部署 Escrow**, 只在新订单的 constructor 里传 Safe 地址即可 (旧订单 arbiter immutable 保持不变)。
- Safe 是合约钱包, 对外暴露 execTransaction, N-of-M 签名后转发 calldata; 从 Escrow 看 msg.sender 就是 Safe 地址。

从技术决策回看业务依据

技术选择	业务依据
EIP-191 personal_sign nonce 走 Redis Lua 原子 chainID 入签名消息 合约 emit PaymentConfirmed listener 写 outbox 而非直推订单 SETNX 72h 事件级幂等 arbiter 走多签路线 WEB3_RPC_URL 未设静默降级	钱包弹窗让用户看到明文，钓鱼可识别 一次性消费，防同链重放 防 mainnet / L2 跨链重放 listener 订一个 event 就能精确对账 与法币通道复用一套对账 reorg / 重连 / 重启业务侧无感 单签平台跑路风险对外无法解释 链不通时主流程仍可用

Web3 的业务价值不在“潮”，在于把“第三方信任”从“中介机构”换成“可审计代码”——这对加密原生 / 跨境 / 大额三类订单都是真痛点。

面试 Q&A (上)

Q1: 链 reorg 把已 release 的资金撤销了怎么办？

A: 风控阈值（确认数）由商家自己设；mainnet 大额走 32 块 = 1 epoch finalize；L2 大额等回 L1 finalize。listener 监听 lg.Removed 跳过 reorg log，订单状态不会被错误推进；若商家已基于浅确认提前发货，arbiter 介入 + 商家走链下追款，平台不兜底。

service/web3/listener.go:218-221

Q2: 用户钱包私钥丢了订单怎么处理？

A: 链上没有“重置密码”。资金在 Funded 态可由仲裁人核实身份后 refund 到新地址；已 Released 给商家则只能走链下补偿。这就是为什么 MVP 阶段单笔 \$100 以内的业务限制——把单笔损失上限封住。

contracts/Escrow.sol:94-100 refund 入口可由 arbiter 触发

Q3: 法币和 Web3 两套通道怎么共存？

A: 路由层并行挂载 /paydown (法币) 和 /paydown/crypto (Web3)，事件层都写入同一条 outbox 总线（事件加 channel 字段：fiat / web3-eth / web3-usdc），对账任务按 order_id join 法币入账 + 链上 confirmed event。下游消费者完全不感知通道差异。

listener.go:14-27 两套路由并行

面试 Q&A (中)

Q5: 客服遇到” 用户付款半小时还没到” 咋办？

A: 分四步问——(1) 是哪条链 (mainnet 拥堵 1-2 小时正常, L2 超 5 分钟才算异常); (2) 钱包侧 tx 是否真的广播了 (用户给 txhash); (3) SRE 后台 grep outbox 是否有 web3.payment.pending (= 后端验签通过); (4) listener 日志该 txhash 是否被消费。三段都能查 = 90% 事故可定位。

repository/cache/web3.go:21-26 Web3PendingTTL=30min internal/payment/service_crypto.go:151-154 SetWeb3Pending

Q6: 为什么 nonce 必须后端发, 前端自己生成不行？

A: 前端可生成的 nonce 后端没法保证唯一性也没法做时效控制; 攻击者拿到一次有效签名后可无限重放。后端 Redis + Lua 原子 GET+DEL 才能保证” 一次性”。

repository/cache/web3.go:60-89 consumeNonceScript

Q7: listener 进程重启会不会丢事件? 业务侧怎么对外解释?

A: 不会。loadLastBlock 从 Redis 拿最后处理块号, catchUp(last+1, head) 批量回放停机期事件, 再叠加 SETNX 72h 幂等, 重启不会重做也不会丢。对外承诺: ”Web3 通道掉单率为 0, 最坏情况是延迟 5 分钟到账”。

面试 Q&A (下)

Q9: 单签 arbiter 多久应该切多签?

A: 单笔 \$100 以内可继续单 EOA; 超过 \$100 或托管总额超 \$10k 强烈建议切 2-of-3 Gnosis Safe; 切换不需重新部署 Escrow (旧订单 arbiter immutable 保持), 新订单 constructor 里传 Safe 地址即可。监管视角也只接受多签——单签平台无法对外解释”为什么这笔 release”。

contracts/Escrow.sol:57-65 constructor; contracts/README.md:102

Q10: 合规 (OFAC) 名单上的钱包地址来 paydown 怎么拦?

A: 在 VerifyAndPark 第 2 段 (验签后、写 outbox 前) 插一个 Chainalysis / TRM Labs 黑名单查询, 命中返回 W3-COMPLIANCE-REJECTED + 客服话术; 不修改合约, 纯链下卡口, 合规可独立审计。路线图阶段, MVP 不接 (业务边界声明)。

internal/payment/service_crypto.go:116-124 验签后是天然过滤点

Q11: RPC 节点全断 30 分钟, 业务怎么办?

A: listener 进入退避重连 (1s → 60s 封顶), 期间链上事件不消费; 新增订单仍可签名挂 pending (验签不依赖 RPC); 法币通道完全不受影响。30 分钟后 RPC 恢复,

listener 自动补齐所有漏掉的事件, business 侧最终一致性 SLA 定位 5 分

代码位置一览（上）

Escrow 合约 + 状态机 *contracts/Escrow.sol:7-113*

immutable 参数 / custom error *contracts/Escrow.sol:8-11; :32-35*

fundWithOrderID 入金 *contracts/Escrow.sol:76-82*

release / refund / dispute *contracts/Escrow.sol:85-107*

钱包验签 EIP-191 *pkg/web3/signature/verify.go:31-82*

Nonce 颁发 + 验签业务 *internal/payment/service_crypto.go:49-160*

签名消息模板 / chainID 防重放 *internal/payment/service_crypto.go:39-46*

Web3 Redis 协议 (nonce/pending) *repository/cache/web3.go:21-101*

Lua 原子 GET+DEL *repository/cache/web3.go:60-89*

代码位置一览（下）

链上 listener 主循环 *service/web3/listener.go:108–230*

catch-up 补块 *service/web3/listener.go:232–248*

reorg 跳过 *service/web3/listener.go:218–221*

事件级幂等 SETNX 72h *service/web3/listener.go:296–326*

last_block 持久化 *service/web3/listener.go:328–352*

退避重连 *service/web3/listener.go:360–377 backoff 1s→60s*

静默降级（env 未设跳过） *service/web3/listener.go:113–117*

Outbox 总线写入 *internal/payment/service_crypto.go:129–144*

API 路由接入位 *internal/payment/routes.go:12–28*

crypto handler *internal/payment/handler_crypto.go:14–52*

合约部署 / 多签建议 *contracts/README.md:1–103*

现状：合约 + binding 在 feat/web3-escrow PR；**MVP** 默认未上链（WEB3 RPC URL 不